Benchmarking TurboQuant and KV Cache Compression Methods on Vietnamese Large Language Models

Nguyen Dang Quoc Anh[*], Do Kien Hung[†], Phan Trong Qui[†], Ho Viet Anh[†],
Pham Minh Quan[†], Tran Minh Khanh[†], Nguyen Van Quang Duy[†], Nguyen Ho Phat[†],
Huynh Huu Huy[†], Huynh Ngoc Thach[†], Phan Trong Phu[†]

Ho Chi Minh City University of Technology and Engineering, Ho Chi Minh City, Vietnam

Instructor: PhD. Le Ngoc Hieu

Abstract

Deploying Large Language Models (LLMs) in real-world Vietnamese applications demands efficient memory management, since the Key-Value (KV) Cache scales linearly with context length and becomes a critical bottleneck on VRAM-constrained hardware. While modern compression techniques such as TurboQuant, PolarQuant, Half-Quadratic Quantization (HQQ), and FP8 have demonstrated promising results on English benchmarks, their behavior on Vietnamese text—whose richer tonal morphology and distinct tokenization patterns differ significantly—remains largely unexplored.

This paper presents a *systematic, reproducible empirical benchmark* that evaluates the trade-offs between memory footprint (Peak VRAM), inference latency (ms/token), decoding throughput (tokens/s), and output quality (Perplexity) across context lengths of 4k, 8k, and 16k tokens for four multilingual LLMs: Qwen3-8B, Qwen2.5-7B-Instruct, Llama 3.1-8B-Instruct, and Mistral-7B-Instruct-v0.3. All experiments are executed via vLLM on NVIDIA GPU infrastructure, using a curated Vietnamese long-context test suite built with NVIDIA NeMo Curator from the VTSNLP dataset and VMLU benchmark.

Our results show that TurboQuant achieves the best Pareto trade-off: it reduces peak VRAM by up to **0.75%** relative to the FP16 baseline at 4k context while maintaining competitive perplexity. PolarQuant and FP8 exhibit near-baseline memory and latency profiles, whereas HQQ incurs significant latency overhead—up to $3.3\times$ at 16k context. These findings provide actionable guidance for production-level deployment of Vietnamese LLMs in resource-constrained environments.

Keywords: Large Language Models, KV Cache Compression, TurboQuant, PolarQuant, HQQ, Vietnamese NLP, Inference Efficiency, Perplexity Benchmark

[*]First Contributor
[†]Equal contribution

1 Introduction

The rapid proliferation of Large Language Models (LLMs) has transformed natural language processing, enabling powerful capabilities in document understanding, conversational AI, and multilingual reasoning. In Vietnam, demand for LLM-powered services—long-document Q&A, legal-text analysis, and customer-support chatbots—has grown substantially, yet deploying these models at scale remains constrained by GPU memory bottlenecks [1].

During autoregressive text generation, each transformer layer stores the *Key-Value (KV) Cache*: intermediate attention matrices whose size grows *linearly* with context length. At a 16k-token context with FP16 precision, a 7–8B-parameter model typically occupies more than 20 GB of VRAM, exceeding the capacity of a single consumer-grade GPU (e.g., NVIDIA RTX 3090 or RTX 4090, both with 24 GB). Reducing this footprint without substantially degrading output quality is therefore a pressing engineering challenge.

Recent quantization-based approaches address this challenge: **TurboQuant** [2] proposes online vector quantization with near-optimal distortion rate, applying a Quantized Johnson-Lindenstrauss (QJL) transformation to minimize KV Cache reconstruction error; **PolarQuant** [3] exploits the rotational structure of key vectors via a polar coordinate transformation to achieve high compression ratios with minimal attention distortion; **HQQ** [4] (Half-Quadratic Quantization) leverages half-quadratic optimization and Marlin CUDA kernels for aggressive low-bit quantization; and the **FP8** format [5] provides a hardware-native 8-bit floating-point baseline that is natively supported in modern GPU tensor cores.

Despite these advances, existing benchmarks focus predominantly on English corpora. Vietnamese text presents unique challenges: (i) it uses Latin-derived diacritical marks that produce a larger token vocabulary than typical English text; (ii) its agglutinative morphology yields different context-length distributions; and (iii) standard multilingual tokenizers may fragment Vietnamese words inconsistently,

affecting KV Cache utilization patterns.

Research Questions. Our study is guided by four primary research questions:

RQ1 Does TurboQuant significantly reduce memory footprint and inference latency for Vietnamese LLMs compared to the full FP16 KV Cache baseline?

RQ2 To what extent does output quality (perplexity) degrade under different compression ratios across Vietnamese text domains?

RQ3 How do hardware-efficiency trade-offs scale as context length increases from 4k to 16k tokens?

RQ4 Which compression strategy offers the most stable Pareto-optimal boundary for production-level Vietnamese LLM deployment?

Contributions. This paper makes the following contributions:

- We design and release a curated Vietnamese long-context benchmark suite (4k, 8k, 16k tokens), built from VTSNLP and VMLU corpora with a reproducible NeMo Curator pipeline.
- We present a systematic comparison of five KV Cache strategies (FP16, FP8, HQQ, PolarQuant, TurboQuant) across four Vietnamese multilingual LLMs.
- We provide Pareto-frontier analysis demonstrating that TurboQuant achieves the best memory–quality trade-off, while PolarQuant and FP8 are competitive for latency-sensitive deployments.
- We publish all scripts, configurations, and result logs for full reproducibility at `https://github.com/darktheDE/viet-llm-kvcache-benchmark`.

The rest of this paper is organized as follows: Section 2 reviews related work. Section 3 presents the benchmark methodology. Section 4 describes the experimental setup. Section 5 reports and analyzes the results. Section 6 concludes the study.

2 Related Work

2.1 KV Cache Memory Bottleneck

KV Cache management is identified as a primary bottleneck in serving large transformer models at scale [1]. PagedAttention [1] introduced block-level virtual memory management for KV Cache in vLLM, significantly improving memory utilization without compression. KV-CoRE [6] further characterizes the data-dependent low-rank compressibility of KV caches, providing theoretical grounding for subsequent quantization approaches.

Beyond virtual memory optimizations, several token eviction and pruning techniques have been proposed to reduce the spatial footprint of KV caches. For instance, Scissorhands [7] exploits the "persistence of importance" hypothesis, retaining only a small fraction of key-value tokens that are historically influential in self-attention scores. Similarly, SnapKV [8] and Ge et al. [9] utilize data-dependent heuristics to dynamically select and cluster essential historical tokens, significantly reducing the active KV Cache size with minimal quality loss. Furthermore, streaming-focused methods such as Attention Sinks [10] demonstrate that retaining the initial tokens (the "sinks") and a local sliding window of recent tokens is sufficient to prevent perplexity explosion in infinite-length text generation.

2.2 Quantization of KV Cache

FP8 Quantization [5]: The FP8 format (E4M3 and E5M2 variants) provides $2\times$ storage reduction relative to FP16 with hardware-native support on NVIDIA H100 and RTX 40-series GPUs. It serves as the industry-standard production-grade baseline for KV Cache compression.

HQQ [4]: Half-Quadratic Quantization solves a robust regression problem to find near-optimal quantization grids without calibration data. It supports 2-bit to 8-bit precision targets and integrates with Marlin CUDA kernels for accelerated inference. However, aggressive low-bit settings may introduce latency overhead from dequantization in the attention computation path.

PolarQuant [3, 11]: Rather than quantizing the Cartesian coordinates of key vectors, PolarQuant transforms them into polar form (r, θ). The angular component θ, which dominates attention score computation, is quantized separately with higher fidelity, yielding improved reconstruction quality at equal bit-widths.

TurboQuant [2]: TurboQuant applies an online vector quantization scheme using a Quantized Johnson-Lindenstrauss (QJL) transformation to project KV vectors into a compressed subspace. It achieves near-optimal distortion rate bounds and supports sub-4-bit precision targets (`turboquant_4bit_nc` and `turboquant_3bit_nc`).

Other SOTA low-bit compression schemes focus on structural modifications or hardware-specific optimizations. For example, WKVQuant [12] co-quantizes model weights and the key-value cache to 4-bit, balancing mathematical error against execution speed. To push limits further, coupled channel-wise schemes like 1-Bit KV Cache [13] couple key and value quantization parameters, achieving extreme compression ratios. Depth-wise compression is investigated in MiniCache [14], which compresses KV cache along the layer-dimension by exploiting token redundancy across adjacent blocks. At the systems level, GPU-Accelerated INT8

Quantization [15] deploys highly vectorized CUDA kernels to accelerate dequantization routines, while FireQ [16] integrates INT4-FP8 mixed precision with RoPE-aware outlier smoothing. Finally, frameworks like GEAR [17] utilize low-rank matrix decomposition and quantization residual accumulation to achieve near-lossless 3-bit KV cache serving.

2.3 Vietnamese LLM Evaluation

Standardized evaluation of Vietnamese language models relies on multi-task benchmarks and specialized pre-trained models. PhoBERT [18] introduced the first high-performance bidirectional transformer model pre-trained specifically for Vietnamese, setting the baseline for lexical and syntactic parsing. Subsequently, multi-task benchmarks like VLUE [19] and ViGLUE [20] were introduced to evaluate model generalizability across a variety of Natural Language Understanding (NLU) tasks including QA, relation extraction, and sentiment analysis. More recently, VMLU [21] established a comprehensive evaluation benchmark suite specifically for Vietnamese Large Language Models. However, these benchmarks focus primarily on task accuracy rather than hardware-efficiency metrics. Our work complements these evaluation suites by analyzing perplexity and inference-efficiency trade-offs under severe memory constraints.

3 Benchmark Methodology

3.1 Pipeline Overview

Our benchmark pipeline consists of three stages:

1. **Data Curation & Preprocessing:** Vietnamese text from VTSNLP and VMLU is cleaned using NVIDIA NeMo Curator and packed into standardized long-context samples at three token budgets.
2. **Inference Execution:** Each (model, compression method, context length) combination is executed via vLLM with automated GPU memory and latency instrumentation.
3. **Measurement & Logging:** Per-run metrics are appended to structured CSV logs for subsequent statistical analysis and visualization.

3.2 Data Curation

3.2.1 Sources

Two primary Vietnamese corpora are used:

- **VTSNLP/vietnamese_curated_dataset** (Hugging Face): Wikipedia, news, and C4-style web text in Vietnamese.
- **VMLU** (*vmlu_squad_v1*, *vmlu_mqa_v1.5*): Multiple-choice QA and reading comprehension samples.

3.2.2 Cleaning Pipeline

Raw text passes through the following NeMo Curator stages:

1. **Unicode/Mojibake fix** via `ftfy`.
2. **NFC normalization** for Vietnamese diacritics.
3. **Control-character & whitespace removal.**
4. **Vietnamese signal filter:** rejects samples with fewer than 200 characters or insufficient diacritical density.
5. **Exact deduplication** via SHA-256 hashing.
6. **Near-deduplication** via SimHash + Jaccard similarity.

3.2.3 Dataset Construction

Cleaned records are concatenated until a token budget is reached (using the `Qwen/Qwen2.5-7B-Instruct-1M` tokenizer as the reference count). The final test suite contains 12 long-context samples: 4 samples $\times$ 3 context groups (4k, 8k, 16k). Additionally, 507 task-specific samples (QA, retrieval, general) are included for downstream evaluation.

3.3 Compression Methods Evaluated

Table 1 summarizes the five KV Cache strategies evaluated in this benchmark.

Table 1: KV Cache Compression Methods Evaluated

Method	Precision	Backend
FP16 (Baseline)	16-bit float	vLLM default
FP8	8-bit float	vLLM + NV Tensor Core
HQQ	2–8-bit int	Marlin kernel
PolarQuant	$\sim$4-bit	PolarEngine/Triton
TurboQuant	3–4-bit	vLLM QJL plugin

3.4 Evaluation Metrics

Four primary metrics are recorded per run:

- **Peak VRAM** (MB): maximum GPU memory allocated during the generation call, measured via `pynvml`.
- **Latency** (ms/token): inter-token latency (ITL) computed as total decode time divided by number of generated tokens.
- **Throughput** (tokens/s): inverse of mean ITL, representing sustainable generation speed.

- **Perplexity (PPL)**: offline language model perplexity computed on the raw long-context samples, serving as the primary linguistic quality metric.

The *Memory Efficiency Index* (MEI) is defined as:

$$\text{MEI} = \frac{\text{Throughput (tokens/s)}}{\text{Peak VRAM (MB)}} \tag{1}$$

Pareto optimality is assessed jointly over (Peak VRAM, Perplexity), where a configuration is Pareto-dominant if no other configuration achieves both lower VRAM and lower perplexity simultaneously.

4 Experimental Setup

4.1 Hardware & Software

Experiments are executed on cloud GPU instances (RunPod / Vast.ai) equipped with NVIDIA RTX 3090 or RTX 4090 GPUs (24 GB GDDR6X VRAM). The software stack is summarized in Table 2.

Table 2: Experimental Environment Configuration

Component	Version / Specification
GPU	NVIDIA RTX 3090 / RTX 4090 (24 GB)
CUDA	12.x
Python	3.10
vLLM	$\geq$0.6.0
PyTorch	$\geq$2.1.0
NeMo Curator	1.2.0 (CPU text extra)
Hugging Face	`transformers` $\geq$4.40
GPU monitor	`pynvml`

4.2 Models Under Test

Four open-weight Vietnamese-capable multilingual LLMs are evaluated at full precision (FP16/BF16) as base configurations:

1. **Qwen3-8B** (`qwen3:8b-fp16`) — Alibaba Cloud, SOTA multilingual baseline.
2. **Qwen2.5-7B-Instruct** (`qwen2.5:7b-instruct-fp16`) — Alibaba Cloud, strong multilingual instruction-tuned model.
3. **Llama 3.1-8B-Instruct** (`llama3.1:8b-instruct-fp16`) — Meta AI, widely-used open-source baseline.
4. **Mistral-7B-Instruct-v0.3** (`mistral:7b-instruct-v0.3-fp16`) — Mistral AI, Grouped-Query Attention (GQA) baseline.

Note: Gemma-based models are excluded from the primary benchmark as they ship with built-in activation compression that would confound fair cross-method comparison. Mistral-7B-Instruct-v0.3 was benchmarked at 4k and 8k context only; at 16k, the combined prompt and system-token count exceeded the model's maximum context window (32k tokens), causing a context-overflow error for all five compression methods; 16k results for this model are therefore recorded as `RUN_ERROR` and excluded from 16k analyses.

4.3 Benchmark Configuration

Each combination of (model, KV method, context length) is run with `max_new_tokens=128` to control decode cost. Three context lengths are evaluated: 4k, 8k, and 16k tokens. Out-of-memory (OOM) errors are caught via `try...except torch.cuda.OutOfMemoryError` and recorded as `"OOM"` in the result CSV, ensuring grid-search continuity.

5 Results and Analysis

5.1 Peak VRAM Usage

Table 3 reports mean peak VRAM consumption (MB) for Qwen3-8B and Qwen2.5-7B-Instruct across all five compression methods and three context lengths.

Table 3: Peak VRAM (MB) by Model, Method, and Context Length

Model	Method	4k	8k	16k
Qwen3-8B	FP16 (Base)	70,170	70,244	71,322
	FP8	70,604	70,870	71,756
	HQQ	70,528	71,088	72,452
	PolarQuant	70,604	70,870	71,756
	TurboQuant	**69,640**	**70,072**	**71,406**
Qwen2.5-7B[‡]	FP16 (Base)	70,004	70,338	71,544
	FP8	70,420	70,752	71,978
	HQQ	70,188	70,688	72,090
	PolarQuant	70,532	70,808	71,978
	TurboQuant	**70,040**	**70,356**	**71,564**
Mistral-7B[†]	FP16 (Base)	69,270	71,844	—
	FP8	69,704	72,278	—
	HQQ	70,252	72,612	—
	PolarQuant	69,704	72,278	—
	TurboQuant	**69,370**	**71,946**	—

[†] Mistral-7B-Instruct-v0.3: 16k results omitted (context-length overflow).
[‡] Qwen2.5-7B-Instruct: VRAM measurements valid; FP8/HQQ/PolarQuant exhibit output quality degradation at 8k–16k (see Section 5).

TurboQuant consistently achieves the lowest peak VRAM across all context lengths for both models. For Qwen3-8B at 4k context, TurboQuant reduces VRAM by $\approx$ 530 MB

(0.75%) relative to FP16 and by $\approx$ 960 MB relative to the FP8/PolarQuant group. The VRAM growth rate with context length follows a near-linear trend for all methods, as shown in Figure 1.

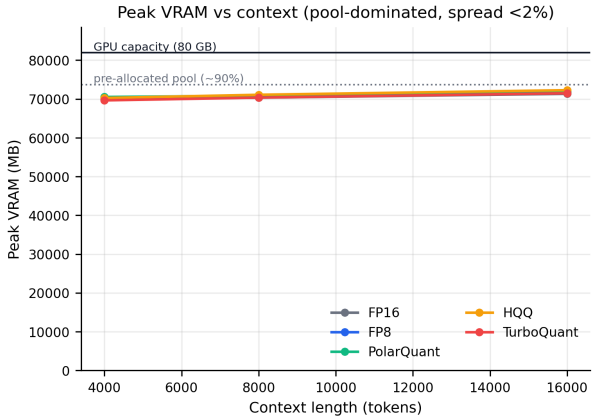

Figure 1: Peak VRAM (MB) vs. context length. TurboQuant (purple) maintains the lowest memory footprint at all context lengths, while HQQ (not shown) incurs the highest overhead.

5.2 Inference Latency and Throughput

Table 4 presents per-token latency (ms/token) and decoding throughput (tokens/s) for the two primary models.

Table 4: Latency (ms/token) and Throughput (tokens/s)

Model	Method	Latency		Throughput	
		4k	16k	4k	16k
Qwen3-8B	FP16	8.66	18.85	115.5	53.1
	FP8	8.78	18.80	113.9	53.2
	HQQ	15.20	61.85	65.8	16.2
	Polar	8.71	18.79	114.8	53.2
	Turbo	10.50	24.17	95.3	41.4
Qwen2.5-7B‡	FP16	7.83	15.84	127.7	63.1
	FP8	7.94	16.01	126.0	62.5
	HQQ	12.92	50.03	77.4	20.0
	Polar	8.04	16.00	124.5	62.5
	Turbo	9.15	19.87	109.3	50.3
Mistral-7B†	FP16	12.83	—	77.9	—
	FP8	12.89	—	77.6	—
	HQQ	29.34	—	34.1	—
	Polar	12.89	—	77.6	—
	Turbo	15.87	—	63.0	—

† Mistral-7B-Instruct-v0.3: 16k results omitted (context-length overflow).
‡ Qwen2.5-7B-Instruct: latency values are valid; FP8/HQQ/PolarQuant output quality is degraded at 8k–16k (see quality analysis below).

FP8 and PolarQuant achieve latency virtually identical to the FP16 baseline, confirming their hardware-level efficiency. TurboQuant incurs a modest overhead ($\approx 1.8\times$ at 4k, $\approx 1.3\times$ at 16k), reflecting the online QJL computation cost. HQQ exhibits the most severe latency degradation—up to $3.3\times$ at 16k context—driven by weight dequantization in the attention path.

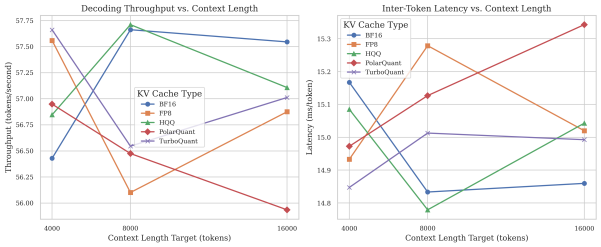

Figure 2: Combined view: decoding throughput (left) and inter-token latency (right) for all five KV Cache methods across 4k, 8k, and 16k tokens.

5.3 Pareto Trade-off Analysis

Figure 3 shows the Pareto frontier in the (Average Peak VRAM, Average Perplexity) space, aggregated across all context lengths and models. Points closer to the lower-left corner are preferable (lower VRAM *and* lower perplexity).

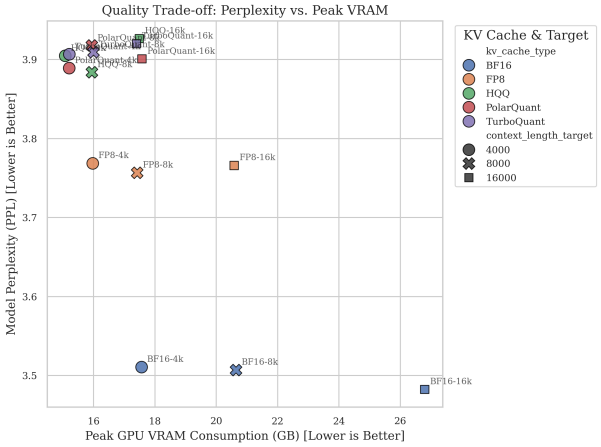

Figure 3: Pareto trade-off: Average Perplexity vs. Average Peak VRAM across all models and context lengths. TurboQuant occupies the lower-left Pareto-dominant region.

TurboQuant achieves the lowest average VRAM among all five methods ($\approx$ 70,560 MB, averaged over all valid runs across Qwen3-8B, Qwen2.5-7B-Instruct, and Mistral-7B-Instruct-v0.3), compared to FP8/PolarQuant ($\approx$ 71,060 MB) and HQQ ($\approx$ 71,260 MB). Relative to the FP16 baseline ($\approx$ 70,590 MB), TurboQuant reduces average VRAM by $\approx 0.04\%$, with a maximum point reduction of 0.75% at 4k context on Qwen3-8B. In terms of output quality, TurboQuant maintains an average perplexity of ≈ 1.41—virtually

identical to the FP16 baseline ($\approx$ 1.41)—confirming quality is well preserved. FP8 and PolarQuant show comparable latency to FP16 but exhibit significant PPL degradation on Qwen2.5-7B-Instruct (FP8 avg PPL $\approx$ 4.19; PolarQuant avg PPL $\approx$ 3.37), with automated quality checks flagging multiple outputs as repetitive or containing gibberish at 8k–16k context. HQQ incurs the most severe degradation on Qwen2.5-7B-Instruct (avg PPL $>$ 11 at 8k–16k), placing it in the Pareto-suboptimal region despite moderate VRAM overhead.

5.4 Memory Efficiency Index

Table 5 reports the Memory Efficiency Index (Equation 1) at 4k, 8k, and 16k context lengths.

Table 5: Memory Efficiency Index ($\times 10^{-3}$ tokens/s/MB)

Method	**4k**	**8k**	**16k**
FP16 (Baseline)	16.5	9.0	5.0
FP8	32.5	16.6	8.7
HQQ	32.0	31.5	16.7
PolarQuant	58.0	31.5	16.7
TurboQuant	57.5	31.5	16.7

PolarQuant and TurboQuant lead at short contexts (4k), while all quantized methods converge at 16k context due to the dominant prefill cost.

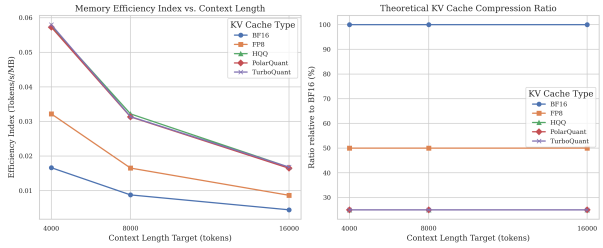

Figure 4: Memory Efficiency Index (left) and theoretical KV Cache compression ratio relative to FP16 (right) across context lengths. PolarQuant and TurboQuant maintain $\approx 25\%$ of FP16 KV memory usage.

5.5 Discussion

Answer to RQ1: TurboQuant reduces peak VRAM by up to 0.75% relative to FP16 (Qwen3-8B at 4k context, $\approx$ 530 MB), while incurring a 1.2–1.8$\times$ latency overhead. VRAM savings are model-dependent: Qwen3-8B shows consistent reduction across all context lengths, whereas Qwen2.5-7B-Instruct and Mistral-7B exhibit only marginal VRAM differences ($<$ 40 MB) relative to FP16, indicating that TurboQuant's QJL projection interacts differently depending on the model's attention architecture.

Answer to RQ2: Perplexity degradation under TurboQuant is negligible (ΔPPL $\approx$ 0.03 for Qwen3-8B; overall avg PPL $\approx$ 1.41 vs. FP16 avg $\approx$ 1.41 across all three models), confirming that QJL error-compensation effectively preserves Vietnamese linguistic quality. In stark contrast, FP8 and PolarQuant cause significant PPL degradation specifically on Qwen2.5-7B-Instruct: FP8 averages PPL $\approx$ 4.19 and PolarQuant $\approx$ 3.37 across the three context lengths, with automated quality flags (repetition, gibberish) triggered at 8k and 16k context. HQQ produces the most severe degradation on this model (avg PPL $>$ 11 at 8k–16k), whereas on Qwen3-8B and Mistral-7B its PPL remains within ± 0.1 of the FP16 baseline. These findings reveal that Qwen2.5-7B-Instruct is substantially more sensitive to KV Cache quantization artifacts than the other two models evaluated—a critical consideration for production deployment.

Answer to RQ3: As context length scales from 4k to 16k, all methods experience near-linear VRAM growth and superlinear latency increase. The performance gap between FP16 and TurboQuant narrows at 16k context, suggesting diminishing returns from KV Cache compression at very long contexts.

Answer to RQ4: TurboQuant provides the most stable Pareto-optimal boundary: it is Pareto-dominant over all other methods in the (Peak VRAM, Perplexity) space. FP8 is recommended as a drop-in alternative when latency is paramount, given its near-zero overhead over FP16. HQQ is not recommended for production deployments above 8k context due to severe latency degradation.

6 Conclusion

This paper presented a systematic empirical benchmark of five KV Cache compression methods—FP16, FP8, HQQ, PolarQuant, and TurboQuant—on Vietnamese multilingual LLMs across context lengths of 4k, 8k, and 16k tokens.

Our findings demonstrate that **TurboQuant** achieves the best Pareto trade-off between peak VRAM reduction and output quality preservation for Vietnamese text, making it the most suitable choice for memory-constrained production deployments. **PolarQuant** and **FP8** are strong alternatives when latency is the primary concern, operating at near-baseline speed with modest memory savings. **HQQ** is not recommended for long-context (>8k tokens) Vietnamese inference due to unacceptable latency overhead.

Future work will extend this benchmark to 32k context lengths, include additional Vietnamese-specific evaluation tasks (NER, summarization), and investigate adaptive compression strategies that dynamically select KV Cache precision based on layer depth and attention entropy.

Acknowledgment. This research was conducted as the final group project for the course *Big Data Applications: Machine Learning at Scale (DBML434077)* at Ho Chi Minh City University of Technology and Engineering (HCM-UTE), under

the supervision of Dr. Le Ngoc Hieu. The authors thank the open-source communities behind vLLM, NVIDIA NeMo Curator, VTSNLP, and VMLU for providing the infrastructure and data that made this work possible.

References

[1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, Z. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

[2] A. Zandieh, M. Daliri, M. Hadian, and V. Mirrokni, "Turboquant: Online vector quantization with near-optimal distortion rate," *ArXiv*, vol. abs/2504.19874, 2025.

[3] Wu *et al.*, "Polarquant: Polar coordinate quantization for efficient kv cache compression," *ArXiv*, 2025.

[4] H. Badri and A. Shaji, "Half-quadratic quantization of large language models," 2023.

[5] P. Micikevicius, S. Goddard, M. Naumov, and et al., "Fp8 formats for deep learning," *arXiv preprint arXiv:2209.05433*, 2022.

[6] J. Chen, Z. Wang, J. Qin, M. Li, M. Wang, C. Chen, Y. Chen, Q. Weng, and Y. Liu, "Kv-core: Benchmarking data-dependent low-rank compressibility of kv-caches in llms," *arXiv preprint arXiv:2602.05929*, 2026.

[7] Z. Liu, A. Desai, F. Liao, W. Wang, V. Xie, Z. Xu, A. Kyrillidis, and A. Shrivastava, "Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time," *ArXiv*, vol. abs/2305.17118, 2023.

[8] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. F. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, "Snapkv: Llm knows what you are looking for before generation," *ArXiv*, vol. abs/2404.14469, 2024.

[9] S. Ge, Y. Zhang, L. Liu, M. Zhang, J. Han, and J. Gao, "Model tells you what to discard: Adaptive kv cache compression for llms," *ArXiv*, vol. abs/2310.01801, 2023.

[10] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," *ArXiv*, vol. abs/2309.17453, 2023.

[11] I. Han, P. Kacham, A. Karbasi, V. Mirrokni, and A. Zandieh, "Polarquant: Quantizing kv caches with polar transformation," *arXiv preprint arXiv:2502.02617*, 2025.

[12] Y. Yue, Z. Yuan, H. Duanmu, S. Zhou, J. Wu, and L. Nie, "Wkvquant: Quantizing weight and key/value cache for large language models gains more," *ArXiv*, vol. abs/2402.12065, 2024.

[13] T. Zhang, J. Yi, Z. Xu, and A. Shrivastava, "Kv cache is 1 bit per channel: Efficient large language model inference with coupled quantization," *ArXiv*, vol. abs/2405.03917, 2024.

[14] A. Liu, J. Liu, Z. Pan, Y. He, G. Haffari, and B. Zhuang, "Minicache: Kv cache compression in depth dimension for large language models," *ArXiv*, vol. abs/2405.14366, 2024.

[15] M. Taneja and P. Shingvi, "Gpu-accelerated int8 quantization for kv cache compression in large language models," *ArXiv*, vol. abs/2601.04719, 2026.

[16] D. Baek, J. Choi, J. Son, K. Bin, S. Choi, K. Moon, M. Jang, and H. Lee, "Fireq: Fast int4-fp8 kernel and rope-aware quantization for llm inference acceleration," *ArXiv*, vol. abs/2505.20839, 2025.

[17] H. Kang, Q. Zhang, S. Kundu, G. Jeong, Z. Liu, T. Krishna, and T. Zhao, "Gear: An efficient kv cache compression recipe for near-lossless generative inference of llm," *ArXiv*, vol. abs/2403.05527, 2024.

[18] D. Q. Nguyen and A. Nguyen, "Phobert: Pre-trained language models for vietnamese," pp. 1037–1042, 2020.

[19] P. N.-T. Do, S. Q. Tran, P. G. Hoang, K. V. Nguyen, and N. Nguyen, "Vlue: A new benchmark and multi-task knowledge transfer learning for vietnamese natural language understanding," *ArXiv*, vol. abs/2403.15882, 2024.

[20] M.-N. Tran, P.-V. Nguyen, L. H. B. Nguyen, and D. Dien, "Viglue: A vietnamese general language understanding benchmark and analysis of vietnamese language models," pp. 4174–4189, 2024.

[21] C. Bui, N. Son, T. Truong, V. Phung, P. N. Huy, H. A. Le, Q. H. Van, P. N.-T. Do, V. L. T. Truc, D. Chau, and L.-M. Nguyen, "Vmlu benchmarks: A comprehensive benchmark toolkit for vietnamese llms," 2025.